

Track Malfunctions in Flatland

Baruch Shteken Lena Friedek

Railway Scheduling, Universität Potsdam
<https://github.com/wug-on-praat/TRailMix>

21st Sep 2026

Track Malfunctions

- real-world issue
- negatively affects punctuality
- reasons: weather, fires, trespassing, repairs, high-frequent use

Malfunctions in Flatland

- solve.py handles train malfunctions.
- Base encoding for scheduling and forwarding information; secondary encoding for rescheduling (Gilkra [1] and Murphy [2]).
- Information forwarded:

- actions as constraints;

```
:- not action(train(0),move_forward,22).  
:- not action(train(1),move_forward,22).  
:- not action(train(1),wait,24).
```

- the malfunction atom;

```
malfunction(1,6,23).
```

- planned_action.

```
planned_action(train(0),move_forward,0).  
planned_action(train(0),move_forward,1).
```

Our Contribution

- create malfunction (randomly/deterministically)
- forwarding additional information
- malfunction logging
- visualization in Flatland

Create a Track Malfunction

```
1 def create_trkmalf(self, timestep, predefined_mals):
2
3     if not predefined_mals:
4         malfunction_ind = random.random()
5
6         if malfunction_ind > 0.99:
7             malfunction_cell = tuple(random.choice(self.grid))
8             malfunction_duration = random.randint(2,20)
9             mal_timestep = timestep
10
11         else:
12             return ([])
13     ...
14     self.track_malfunctions.append((malfunction_cell,
15                                     malfunction_duration))
16     self.track_record.append((malfunction_cell, mal_timestep,
17                               malfunction_duration))
18     return([(malfunction_cell, malfunction_duration)])
```

Additional Forwarded Information

- track malfunction

```
track_malfunction((17, 19), 21, 13).
```

- position constraint rules

```
:- position( _, (17, 19), 14, _).
```

```
:- position( _, (17, 19), 15, _).
```

- planned position

```
planned_position(0, (20, 3), 15, s).
```

```
planned_position(1, (28, 25), 15, n).
```

- current position

```
current_position(0, (19, 3), 14, s).
```

```
current_position(1, (29, 25), 14, n).
```

Secondary Encoding

- add figures for base and secondary
-

Rescheduling

The basis of our secondary encoding is to determine which trains are affected by the malfunction and reschedule only them unless there is a collision and then reschedule also the trains affected by the collision

- Check if there is a consequence

```
consequence(train(ID), P, T) :- planned_position(ID, P, T, _),  
track_malfunction(P, Dur_mal, T_mal), Dur_mal + T_mal >= T.
```

- Use as many planned trains as possible

```
{planned(ID)}:- train(ID), not consequence(train(ID), _, _).  
#maximize { 1@3, ID : planned(ID)}.
```

- Recalculate schedule for unplanned trains

```
calculate(ID) :- consequence(train(ID), _, _).  
calculate(ID) :- train(ID), not planned(ID).
```


Experiment 1 - Heuristic vs. no heuristic

- Does the Heuristic improve solving?
- prefer planned trains:

```
#heuristic planned(ID): train(ID). [100, true]
```

- **Results - Yes!**

- small environments: no difference
 - big/dense environments: heuristic significantly faster (especially in no consequence scenarios - 4 mins)
- > less re-proving necessary

Experiment 2 - Base vs. Secondary

- Is Secondary really better than Redo Planning (Waldow [3])?
- **Results - Yes!**
 - small environments: little/ no difference
 - big/dense environments: secondary encoding significantly faster (2 mins in no consequence)
 - > successful use of forwarded information

The encodings are different only in their optimization rule prioritization

- First Plan Prioritization

```
#maximize { 1@3, ID : planned(ID)}.
```

```
#minimize { 1@2, ID, T : action(train(ID), _, T) }.
```

- Time Prioritization

```
#maximize { 1@2, ID : planned(ID)}.
```

```
#minimize { 1@3, ID, T : action(train(ID), _, T) }.
```

Experiment 3 - Base vs. Time Prio

- Is Secondary with Time Prioritization better than Redo Planning?
- **Results - Yes??**
 - no consequence: base is faster (40 s in large, dense environment) or similar
 - other consequences: secondary is significantly faster (50% in a complex scenario)
- > Time Prio struggles with direct translation of original plan

Experiment 4 - First Plan vs. Time Prioritization

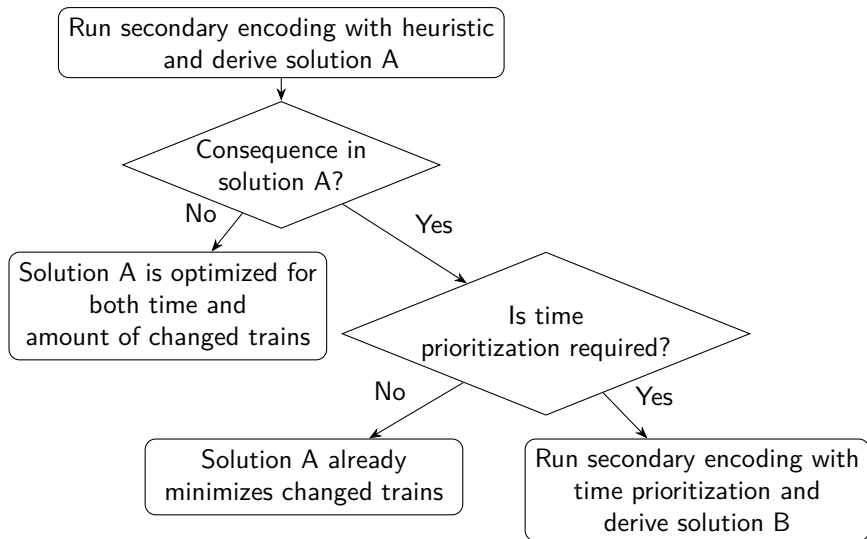
- Is Time Prioritization better than First Plan Prioritization?
- **Results - No!**
 - first plan encoding is significantly faster in almost all environments and scenarios
 - similar performance in small environments
 - > First Plan Prio is faster BUT...

Purple Train



Figure: Qualitative differences between encodings.

Discussion



Future Work

- add figures for base and secondary
-

References



Gili i Bueno, G., Kratzsch, F.: Flatland: Malfunctions research on train training (2025),
<https://github.com/krr-up/railway-research/blob/main/railway-course/2024-wise/gilkra2025a.pdf>



Murphy, K.R.: Flatland (2026),
<https://github.com/krr-up/flatland>, Accessed: 17.08.2026



Waldow, I.: Intelligent malfunction handling in large railway systems (2025), <https://github.com/krr-up/railway-research/blob/main/theses/waldow2025a.pdf>